

# Brick ตรวจจับวัตถุ (Object Detection Brick)

Brick นี้มีอินเทอร์เฟซภาษา Python สำหรับตรวจจับวัตถุในภาพที่กำหนดไว้

## ภาพรวม (Overview)

Object Detection Brick ช่วยให้สามารถ:

- ตรวจจับวัตถุในภาพได้ ทั้งจากไฟล์ภาพภายในเครื่องหรือจากกล้องโดยตรง
- ระบุตำแหน่งของวัตถุที่ตรวจจับได้ในภาพโดยใช้กรอบสี่เหลี่ยม (bounding boxes)
- รับค่าความมั่นใจ (confidence value) ของการตรวจจับแต่ละวัตถุและชื่อป้ายกำกับ (label)

## คุณสมบัติ (Features)

- ทำการตรวจจับวัตถุแบบเรียลไทม์บนภาพนิ่ง
- แสดงผลกรอบสี่เหลี่ยม, ป้ายชื่อวัตถุ (class label), และค่าความมั่นใจ (confidence score) สำหรับแต่ละวัตถุที่ตรวจพบ
- รองรับหลายรูปแบบไฟล์ภาพ เช่น JPEG, JPG, และ PNG (ค่าเริ่มต้น: JPG)
- อนุญาตให้ปรับค่าเกณฑ์ความมั่นใจในการตรวจจับและค่าการกรองซ้อน (Non-Maximum Suppression, NMS)
- สามารถเชื่อมต่อได้ง่ายกับภาพจากไลบรารี PIL หรือข้อมูลภาพแบบ byte stream โดยตรง

## ตัวอย่างโค้ดและวิธีใช้งาน (Code example and usage)

```
import os #นำเข้าออปเจ็ค os เพื่อใช้ตรวจสอบไฟล์ในระบบ เช่น ตรวจสอบว่ามีไฟล์อยู่จริงหรือไม่
from arduino.app_bricks.object_detection import ObjectDetection #นำเข้า ObjectDetection class จาก Arduino App Bricks สำหรับตรวจจับวัตถุในภาพ
from arduino.app_utils import * #นำเข้าฟังก์ชันหรือยูทิลิตี้อื่น ๆ จาก Arduino App Utils (เช่น ฟังก์ชันช่วยเหลือต่าง ๆ)
import time #นำเข้าออปเจ็ค time เพื่อใช้หน่วยเวลา เช่น time.sleep()
```

```
def main(): #ประกาศฟังก์ชันหลัก main() ซึ่งจะเป็นจุดเริ่มต้นของโปรแกรม
    img_path = "Man.jpg" #กำหนดชื่อไฟล์รูปภาพที่จะนำมาตรวจจับวัตถุ แนะนำว่าควรเป็นไฟล์นามสกุล .jpg หรือ png

    if not os.path.exists(img_path): #เขียนเงื่อนไขแจ้งเตือนว่ามีรูปภาพไฟล์ดังกล่าวจริงหรือไม่ถ้าไม่ให้แสดงข้อความ
        print(f"ไม่เจอรูปภาพนะครับ: {img_path}") #แสดงข้อความ
        return #ออกจากฟังก์ชันทันทีออกจากฟังก์ชัน main() ทันที
    print("เจอรูปภาพแล้ว ทำงานต่อได้") #ถ้าเจอไฟล์จะทำงานต่อ

    object_detection = ObjectDetection() #สร้างอ็อบเจกต์ของคลาส ObjectDetection เพื่อเรียกใช้งานฟังก์ชันตรวจจับวัตถุใน Brick ของ Object Detection
```

```
    with open(img_path, "rb") as f: #เปิดไฟล์รูปภาพในโหมดอ่านแบบไบนารี (r =อ่านไฟล์ (read), b = อ่านแบบไบนารี (binary) โดยการอ่านแบบไบนารีนี้จะใช้กับไฟล์รูปภาพ, ไฟล์เสียง, หรือไฟล์ที่ไม่ใช่ข้อความ) คำสั่ง with เป็นคำสั่งเปิดไฟล์แล้วปิดอัตโนมัติเมื่อทำงานเสร็จ และ f เป็นตัวแปรแทนไฟล์ เราสามารถใช้ f.read() หรือ f.write() ได้ดังคำสั่งข้างล่าง
        frame = f.read() #อ่านข้อมูลไฟล์ทั้งหมดเก็บไว้ในตัวแปร frame
```

out = object\_detection.detect(frame) #ใช้เมธอด detect() ตรวจสอบวัตถุในภาพ โดยผลลัพธ์จะถูกเก็บในตัวแปร out (จะมีข้อมูลประเภทวัตถุ ความมั่นใจ และพิกัดกล่องรอบวัตถุ)

if out and "detection" in out: #ตรวจสอบว่า out ไม่ใช่ None หรือค่าว่าง และ ตรวจสอบว่า key "detection" มีอยู่ใน dictionary out หรือเปล่า ถ้ามีทั้งคู่ให้เข้าไปทำคำสั่ง for ด้านล่างต่อแต่ถ้าตรวจสอบแล้วเป็นค่าว่างหรือ None จะไม่เข้า loop for ด้านล่างเลย

for i, obj\_det in enumerate(out["detection"]): #ถ้ามีผลลัพธ์จาก detect() และมี key "detection" อยู่ใน out จะทำต่อบรรทัดนี้ โดยคำสั่ง out["detection"] ใช้ดึง list ของวัตถุที่ตรวจจับได้จาก dictionary เช่น ตัวอย่างด้านล่างดังรูปที่ 1 จะได้ list 2 อ็อบเจกต์ และคำสั่ง enumerate(...) ใช้เพื่อสร้าง ลำดับหมายเลข (index) ให้กับแต่ละวัตถุใน list โดย i คือ index เริ่มจาก 0 และ obj\_det คือ dictionary ของวัตถุแต่ละตัว

```
print(f'[{i+1}] Object: {obj_det.get('class_name')}, Confidence: {obj_det.get('confidence')}')
```

```
print(f'Bounding box: {obj_det.get('bounding_box_xyxy')}\n')
```

#ตรวจสอบว่ามีผลลัพธ์การตรวจจับหรือไม่ ถ้ามี ให้วนลูปแต่ละวัตถุและพิมพ์: ชื่อประเภทของวัตถุ (class\_name) ความมั่นใจในการตรวจจับ (confidence) และข้อมูลพิกัดกล่องรอบวัตถุ (bounding\_box\_xyxy) โดยคำสั่ง obj\_det.get('class\_name') คือ การดึง ชื่อวัตถุ และ obj\_det.get('confidence') คือ การดึง ค่าความมั่นใจ (0-1)

out\_image = object\_detection.draw\_bounding\_boxes(frame, out) #วาดกล่องรอบวัตถุนภาพต้นฉบับ

```
print("ตรวจสอบภาพเสร็จจะครับเจ้านาย")
```

```
time.sleep(2)
```

```
if __name__ == "__main__":
```

```
    main()
```

#คำสั่ง if ใช้เพื่อกันไม่ให้ไฟล์อื่น Import ฟังก์ชัน main() ไปใช้ และในขณะเดียวกันใช้เพื่อต้องการให้ฟังก์ชัน main() รันเพียงรอบเดียว

โครงสร้างของตัวแปร out จะมีลักษณะเป็น dictionary ซึ่งมี key “detection” อยู่ข้างในถ้าตรวจจับได้

```
out = {
    "detection": [
        {"class_name": "person", "confidence": 0.95, "bounding_box_xyxy": [125, 179, 480, 335]},
        {"class_name": "dog", "confidence": 0.88, "bounding_box_xyxy": [50, 100, 200, 250]}
    ]
}
```

รูปที่ 1 Dictionary ภายในตัวแปร out มี list อยู่ 2 อ็อบเจกต์

ตัวอย่างการทำงาน:

```
python
```

```
for i, obj_det in enumerate(out["detection"]):  
    print(i, obj_det)
```

ผลลัพธ์:

```
bash
```

```
0 {'class_name': 'person', 'confidence': 0.95, 'bounding_box_xyxy': [125, 179, 480, 335]}  
1 {'class_name': 'dog', 'confidence': 0.88, 'bounding_box_xyxy': [50, 100, 200, 250]}
```

รูปที่ 2 การทำงานของคำสั่ง enumerate สำหรับตัวแปร out

## สรุปการทำงานของ flow

if out and "detection" in out:

```
for i, obj_det in enumerate(out["detection"]):  
    print(f"[{i+1}] Object: {obj_det.get('class_name')}, Confidence: {obj_det.get('confidence')}")  
    print(f"Bounding box: {obj_det.get('bounding_box_xyxy')}\n")
```

คือ

1. ตรวจสอบว่าผลลัพธ์การตรวจจับมีหรือไม่ (if out and "detection" in out)
2. ถ้ามี → วนลูปแต่ละวัตถุใน list out["detection"]
3. สำหรับแต่ละวัตถุ → พิมพ์
  - ลำดับที่
  - ชื่อประเภทวัตถุ
  - ความมั่นใจ
  - พิกัดกล่องรอบวัตถุ

## คลาส ObjectDetection

```
class ObjectDetection(confidence: float)
```

โมดูลสำหรับการตรวจจับวัตถุในภาพโดยใช้โมเดลแมชชีนเลิร์นนิงที่ระบุ

โมดูลนี้จะประมวลผลภาพอินพุตและส่งคืน:

- กรอบขอบเขต (Bounding boxes) สำหรับวัตถุที่ตรวจจับได้
- ป้ายชื่อคลาส (Class labels) ที่สอดคล้องกัน
- คะแนนความเชื่อมั่น (Confidence scores) สำหรับการตรวจจับแต่ละครั้ง

## เมธอด (Methods)

```
detect_from_file(image_path: str, confidence: float)
```

ประมวลผลไฟล์ภาพในเครื่องเพื่อตรวจจับและระบุวัตถุ

### พารามิเตอร์ (Parameters)

- image\_path: เส้นทางไปยังไฟล์ภาพในระบบไฟล์ของเครื่อง
- confidence: เกณฑ์ความเชื่อมั่นขั้นต่ำสำหรับการตรวจจับ ค่าเริ่มต้นคือ None (ใช้ค่าเริ่มต้นของโมดูล)

### ค่าที่ส่งคืน (Returns)

- (dict): ผลลัพธ์การตรวจจับที่ประกอบด้วยชื่อคลาส, ความเชื่อมั่น, และกรอบขอบเขต

## เมธอด (Methods)

```
detect(image_bytes, image_type: str, confidence: float)
```

ประมวลผลรูปภาพที่อยู่ในหน่วยความจำ (in-memory image) เพื่อตรวจจับและระบุวัตถุ

### พารามิเตอร์ (Parameters)

- image\_bytes: สามารถเป็นข้อมูลไบนารีดิบ (raw bytes) (เช่น จากไฟล์หรือสตรีม) หรือเป็นรูปภาพ PIL ที่โหลดไว้ล่วงหน้า

- image\_type, ค่าเริ่มต้น="jpg": รูปแบบไฟล์ภาพ ('jpg', 'jpeg', หรือ 'png') จำเป็น หากใช้ข้อมูลไบต์ดิบ ค่าเริ่มต้นเป็น 'jpg'
- confidence: เกณฑ์ความเชื่อมั่นขั้นต่ำสำหรับการตรวจจับ ค่าเริ่มต้นคือ None (ใช้ค่าเริ่มต้นของโมดูล)

### ค่าที่ส่งคืน (Returns)

- (dict): ผลลัพธ์การตรวจจับที่ประกอบด้วยชื่อคลาส, ความเชื่อมั่น, และกรอบขอบเขต

### เมธอด (Methods)

`draw_bounding_boxes(image: Image.Image | bytes, detections: dict)`

วาดกรอบขอบเขตลงบนรูปภาพที่ล้อมรอบวัตถุที่ตรวจจับได้ โดยใช้ไลบรารี PIL

#### พารามิเตอร์ (Parameters)

- image: รูปภาพอินพุตที่จะใส่คำอธิบายประกอบ (annotate) สามารถเป็นวัตถุรูปภาพ PIL หรือข้อมูลไบต์ดิบของรูปภาพ
- detections: ผลลัพธ์การตรวจจับที่ประกอบด้วยป้ายชื่อวัตถุและกรอบขอบเขต

### ค่าที่ส่งคืน (Returns)

รูปภาพพร้อมกรอบขอบเขตและจุดสำคัญ (key points) ที่วาดแล้ว หากไม่มีการตรวจจับหรือรูปภาพไม่ถูกต้อง จะเป็นค่า None

### เมธอด (Methods)

`process(item)`

ประมวลผลรายการเพื่อตรวจจับวัตถุในรูปภาพ

เมธอดนี้รองรับรูปแบบอินพุตสองแบบ:

- สตริงเส้นทางไปยังไฟล์ภาพในเครื่อง
- ดิกชันนารีที่มีข้อมูลไบต์ดิบของรูปภาพภายใต้คีย์ 'image' และเลือกได้ว่าจะให้คีย์ 'image\_type' (เช่น 'jpg', 'png') หรือไม่

#### พารามิเตอร์ (Parameters)

- item: เส้นทางไฟล์ (str) หรือดิกชันนารีที่มีคีย์ 'image' และ 'image\_type' (dict) โดยที่ 'image\_type' เป็นทางเลือก ในขณะที่ 'image' มีข้อมูลรูปภาพเป็นไบต์

คำว่า "overlap" และ "NMS threshold" เป็นแนวคิดที่สัมพันธ์กันอย่างมากในขั้นตอนหลังการประมวลผล (Post-processing) ของการตรวจจับวัตถุ (Object Detection) เพื่อกำจัดกล่องล้อมรอบวัตถุที่ซ้ำซ้อนกัน

### 1. overlap (ค่าการทับซ้อน)

ในบริบทของการตรวจจับวัตถุ overlap คือการวัดระดับการทับซ้อนระหว่างกล่องล้อมรอบวัตถุสองกล่อง มักใช้หน่วยวัดที่เรียกว่า Intersection over Union (IoU)

IoU คืออะไร: คือค่าที่คำนวณจาก (พื้นที่ส่วนที่ทับซ้อนกัน) หารด้วย (พื้นที่รวมทั้งหมดของกล่องทั้งสอง) โดยค่าจะอยู่ระหว่าง 0 ถึง 1

IoU = 1: หมายถึงกล่องทั้งสองทับซ้อนกันสมบูรณ์แบบ

IoU = 0: หมายถึงกล่องทั้งสองไม่มีส่วนทับซ้อนกันเลย

**บทบาท:** ในฟังก์ชัน `object_detection.detect()` พารามิเตอร์ overlap (เช่น `overlap=0.5`) เป็นการกำหนดเกณฑ์ (Threshold) สำหรับการตัดสินใจว่ากล่องล้อมรอบวัตถุนั้น ทับซ้อนกันมากเกินไป จนถือว่าเป็นกล่องซ้ำซ้อนกันหรือไม่

### 2. NMS threshold (เกณฑ์ Non-Maximum Suppression)

NMS ย่อมาจาก Non-Maximum Suppression (การระงับค่าที่ไม่สูงสุด) ซึ่งเป็นอัลกอริทึมที่ใช้หลังจากการตรวจจับ เพื่อเลือกกล่องล้อมรอบวัตถุที่ดีที่สุดเพียงกล่องเดียวจากกล่องที่ซ้ำซ้อนกันหลายกล่อง

วัตถุประสงค์: "เพื่อกรองรอบซ้ำ" หรือเพื่อลบกล่องล้อมรอบวัตถุที่ซ้ำซ้อนกัน (ซึ่งมักเกิดจากการที่โมเดลคาดการณ์หลายกล่องรอบวัตถุเดียวกัน)

ขั้นตอนการทำงานของ NMS:

- จัดเรียง: เรียงลำดับกล่องล้อมรอบวัตถุทั้งหมดตามคะแนน confidence (ความเชื่อมั่น) จากมากไปน้อย
- เลือกกล่องที่ดีที่สุด: เลือกกล่องที่มีคะแนน confidence สูงสุดมาเป็นผลลัพธ์สุดท้าย
- กรองกล่องซ้ำ: เปรียบเทียบกล่องที่ดีที่สุดกับกล่องที่เหลือทั้งหมด โดยการคำนวณ IoU
- ระบุ: ถ้าค่า IoU (overlap) ระหว่างสองกล่องมีค่าสูงกว่า NMS threshold ที่กำหนด (เช่น 0.5) แสดงว่ากล่องเหล่านั้นทับซ้อนกันมากเกินไปและน่าจะตรวจจับวัตถุเดียวกัน กล่องที่มีคะแนน confidence ต่ำกว่าจะถูกกำจัดออก (suppressed)
- ทำซ้ำ: ทำซ้ำขั้นตอนเดิมกับกล่องที่เหลือ จนกระทั่งกล่องทั้งหมดถูกประมวลผลหมด